07. Deploy a Custom Container with Portenta X8 Manager

This tutorial will show you how to create and upload your custom container to your Portenta X8.

Author · Benjamin Dannegård Last revision · 09/25/2024

Overview

In this tutorial, we will create a Docker container for the Arduino Portenta X8. We will start by building a Docker image, which includes all necessary code and dependencies. Then, we will show how to deploy this image to the Portenta X8 and run it as a container. This process involves using ADB for device communication to manage containers on the Portenta X8.

Goals

- Learn how to create and understand Docker images for the Portenta X8
- Learn how to deploy and run containers on the Portenta X8

Required Hardware and Software

ADB: Check how to connect to your Portenta X8

USB-C® cable (USB-C® to USB-A cable)

Arduino Cloud Subscription

Arduino IDE 1.8.10+, Arduino IDE 2, or Arduino Cloud Editor

Instructions

A Docker container operates with an isolated filesystem derived from its image. The container's image acts as a blueprint, providing a custom filesystem containing all necessary components, such as dependencies, configurations, scripts, and binaries. It also includes settings like environment variables, default commands, and other metadata to ensure the container runs as intended.

Container File Structure

Organize the essential files within a directory named **x8-custom-test** to prepare for container creation. This directory should include:

docker-build.conf: Configuration for build specifics, including tests to verify the container's functionality.

docker-compose.yml: YAML file for defining and running multi-container Docker applications.

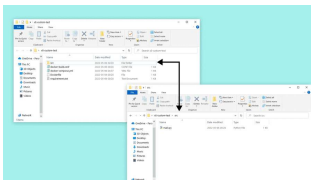
Dockerfile: A script with commands to assemble the image.

the application.

src folder: A directory for source code.

main.py: The main Python® script, located inside the src folder.

This organization eases a structured approach to building the Docker container. The complete folder should look as the following structure:



Folder structure for container

Docker-build.conf

This file specifies commands for basic validation tests within the container. Our setup defines a simple test to ensure the container's Python® environment is operational:

```
1  TEST_CMD="python3 --help"
```

Docker-compose.yml

The **docker-compose.yml** file stages the configuration of your application's services. This example defines a single service named **x8-custom-test**, configuring it with specific runtime properties such as restart policy, user permissions, and

this case, *blob-opera:latest*, which will be built locally if it does not exist in the Docker registry.

```
version: '3.6'

services:
  x8-custom-test:
    image: blob-opera:la
    restart: always
    tty: true
    read_only: true
    user: "63"
    tmpfs:
    - /run
    - /var/lock
    - /var/log
    - /tmp
```

Dockerfile

The **Dockerfile** is a blueprint for building a Docker image, containing all the instructions (`FROM`, `COPY`, `COMMAND`, `ENTRYPOINT`, etc.) and detailing all steps from the base to the final image. It specifies the base image to use, the working directory, dependencies to install, and the command to run on container startup.

It sets up a Python® environment, installs dependencies from *requirements.txt*, and ensures the *main.py* script runs when the container is created.

```
FROM python:3-alpine3.15

# Set our working direct
WORKDIR /usr/src/app

# Copy requirements.txt
COPY requirements.txt re

# pip install python dep
RUN pip install -r requi

# This will copy all fil
COPY ./src/main.py ./

# Enable udevd so that p
ENV UDEV=1

# main.py will run when
CMD ["python","-u","main
```

Requirements.txt

This file lists the Python® packages required by your application, ensuring all dependencies are installed during the image build process. For this example, *Flask* is the required dependency, pinned to a specific version for consistency, reliability, or preference.

```
Flask==0.12.3
```

Source

Here we will keep the source code of the app you want to run in the container or a startup script. We will create a **main.py** file in this folder. This script will print "Hello World!" in the CLI window.

This section is dedicated to the

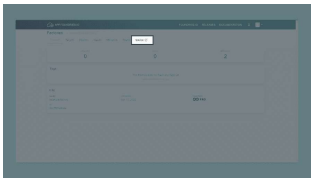
src folder, we will create a file named *main.py*. This script, using *Flask*, will display `"Hello World!"` in the CLI window.

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello World!'

if __name__ == '__main__'
    app.run(host='0.0.0.0
```

Uploading the Container Folder

Begin by resetting your board to its Factory settings as outlined in the Portenta X8 **Out-of-the-box experience from the User Manual**.

Next, upload the **x8-custom-test folder** to a repository within the Factory environment. This repository, typically named *containers.git*, can be found on your Factory page under *Source*. Use the repository's URL for the following operations.



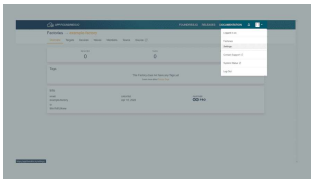
Source on Foundries.io Factory page

Where to find container.git

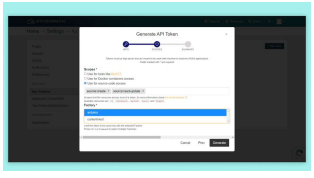
Container.git page

To pull or push repositories, you have to generate an API key. This is done by going to the user settings on the Factory page. Click on the user drop-down menu, enter the tokens page, and follow the steps to create a new API key. When creating the API key, please select the *Use for source code access* option and the correct Factory for which you want to use the key.

This token will be the password for all git operations, while the username can be anything except an empty string.



User settings on your Factory page



Token creation section in Foundries.io

Use the following command in git on your machine. To get the repository on your machine, replace *YOUR_FACTORY* with the name of your Factory. After cloning the repository, the *-b* parameter specifies a branch to checkout. Running this command will get the container repository, where we will put our folder.

```
1  git clone https://source.
```

After cloning, add the **x8-custom-test** folder to this repository and push it with git.

When you have put the folder into the git folder, use `git status` to review changes; it will show the unadded changes in red. Then use `git add` to add the changes you want to your `git commit`. Then use `git commit` and `git push` to finally push the changes to the repository. Successfully pushing to "containers.git" triggers a new build in your FoundriesFactory, visible on the *Targets* page.

Building and Running the Container

Once the build process is complete, it may take up to 10 minutes for your device to receive and apply the update over-the-air. You can monitor the update's progress through the *Devices* tab in your FoundriesFactory interface. After the update, the **x8-**

custom-test folder will be on your device, ready for the next steps.

To build the container, use the `docker build` command in your Dockerfile's directory. The `--tag` option allows us to assign a version or name to the build, providing easier management of different container versions.

```
docker build --tag "x8-cu
```

You can start the container using `docker run` with the built container image. Here, the `--user` flag is used to set the user identity (UID) for the container's process, as specified in the *docker-compose.yml* file.

```
docker run -it --rm --use
```

This command starts the container interactively (`-it`), removes it after exit (`--rm`), and runs it under the specified user. The container will run with the settings and application defined in your *Dockerfile* and *Docker-compose* configurations.

Using Docker Compose

For scenarios involving complex configurations, `docker compose` offers a streamlined approach to managing containerized applications. It removes the need for extensive command-line arguments

navigating to the container's directory:

```
cd /home/fio/x8-custom-te
```

Using `docker compose`, the following command starts your application and sets it up as a `systemd` service, ensuring its persistence across reboots. As a result, your application will automatically start upon system startup:

```
docker compose up --detac
```

To stop the `docker compose` application, use the command below:

```
docker compose stop
```

Deploying with Docker Hub

Docker Hub provides an alternative deployment method by hosting your container images on its platform. Start by creating a **Docker Hub account** and setting up a repository for your container. Once your repository is ready, use the following command to upload your container image:

```
docker push HUB_USERNAME/
```

Your custom container image will be available in your Docker Hub repository and accessible from any location with internet connectivity. To deploy this image to your Portenta X8, connect the device via ADB and execute the following commands. First, enter the device's shell:

```
adb shell
```

Then, pull and deploy the container image:

```
docker pull x8-custom-tes
```

This command retrieves the container image, allowing you to deploy and run the container on your Portenta X8.

> For detailed instructions on creating and managing Docker Hub repositories for your custom Portenta X8 containers, refer to the official **Docker Hub Documentation**.)

Conclusion

In this tutorial, we have outlined how to structure a Docker container for the Portenta X8, describing the

Foundries.io's Factory for streamlined deployment and highlighted building and running the container on the device. Lastly, we introduced docker compose as a tool for testing containers, emphasizing speed and convenience.

Next Steps

To get a better understanding of how to manage containers with Docker, take a look at our **Managing Containers with Docker on Portenta X8**. This tutorial will show some useful commands for the docker service and ADB or SSH.

Troubleshooting

Here are some errors that might occur in the process of this tutorial:

> Make sure you have followed our other tutorials that show how to set up the Portenta X8 with **Out-of-the-box experience from the User Manual**
>
> If you are having issues with the adb shell, don't forget to try and use `sudo` and `su`

---

this page
here.

Was this article helpful?